
Large Language Model Inference Systems

From one request to a production serving stack

Shuhao Zhang

Huazhong University of Science and Technology

China-Central Asia Advanced Computing Training Program
August 4, 2026

A concrete story: one chatbot question

User sees

- Type a question.
- Wait for the first word.
- Watch tokens stream out.
- Abandon the task if the service feels slow.

System does

- Tokenize and validate the request.
- Queue it with other users' requests.
- Run prefill to create KV cache.
- Run decode for many iterations.
- Stream output and recycle memory.

LLM serving is an online stateful system.

Two-hour roadmap

Session 1: serving fundamentals

- Service quality and request lifecycle
- Prefill, decode, and user-visible metrics
- Scheduling, continuous batching, and KV state

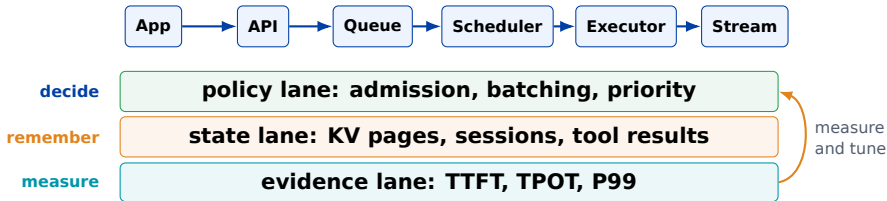
Session 2: modern system challenges

- Long context, RAG, MoE, agents, and accelerators
- Benchmark evidence and open engineering workflow
- Concepts, open questions, and readings

Guiding thread

Follow the request, follow the state, and measure the service quality.

The big picture



- Many optimizations do not happen inside a single kernel.
- They happen at control points: queueing, batching, caching, memory movement, and validation.

Part I. Serving System Mental Model

From user-visible service quality to the lifecycle of one request

From model capability to service capability

Models changed

- Long context
- MoE
- Multimodal input
- Tool-using agents

Services changed

- Chat APIs
- Enterprise assistants
- Scientific agents
- Multi-tenant platforms

Metrics changed

- First-token delay
- Tail latency
- Cost per request
- SLO satisfaction

Takeaway

A good model can still become a bad product if the serving system is slow, unstable, or too expensive.

Throughput alone is incomplete

Server A

- High average throughput.
- Large batches.
- Some users wait a long time before the first token.

Server B

- Slightly lower throughput.
- Smaller waiting time.
- More requests meet the latency target.

Better service depends on the objective: throughput, first-token latency, tail latency, or SLO compliance.

Training systems vs. inference systems

Dimension	Training	Inference serving
Workload	Large stable batches	Online arrivals, mixed request lengths
Objective State	Time-to-train, scaling efficiency Parameters, gradients, optimizer states	TTFT, TPOT, P99, cost, SLO KV cache, queues, sessions, logs
Control point	Step / micro-batch	Request / iteration / token
Failure mode	Slow convergence, low utilization	Jitter, head-of-line blocking, cache pressure

Working view

Inference serving behaves like a **stateful online service**, not a static model call.

Four recurring checks

1. **How are requests organized?** Queueing, batching, priorities, and SLOs.
2. **How is state managed?** KV allocation, reuse, migration, and eviction.
3. **Which execution path is used?** Kernels, graph replay, backends, and fallbacks.
4. **What evidence supports the result?** Workload, baseline, metadata, and metrics.

Request

State

Execution

Evidence

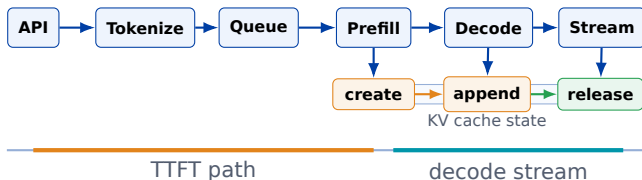
The request lifecycle

1. API receives a prompt and generation parameters.
2. Tokenizer converts text into token IDs.
3. Scheduler admits the request into a waiting queue.
4. Prefill processes the prompt and creates the first KV cache.
5. Decode repeatedly generates new tokens.
6. The server streams output and releases resources.

Why this view matters

A clear performance claim names the stage it changes and the cost it moves.

Lifecycle diagram



- TTFT is dominated by queueing and prefill.
- Later token latency is dominated by decode scheduling and KV residency.

Prefill and decode are different workloads

Prefill

- Reads the whole prompt.
- More compute intensive.
- Determines first-token latency.
- Becomes expensive with long context and RAG.

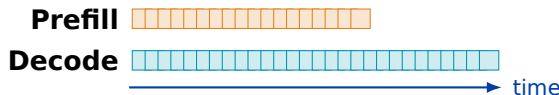
Decode

- Generates one or a few tokens per iteration.
- Lighter per step, but repeated many times.
- Relies on KV cache staying alive.
- Sensitive to scheduling jitter.

They expose different bottlenecks and need different optimization paths.

Example request: “Explain transformer attention”

- Prompt: 600 tokens.
- Output: 120 tokens.
- Prefill: one relatively heavy pass.
- Decode: 120 small iterations.



Observation

A user feels both the wait before the first token and the smoothness of the stream afterwards.

Service metrics

Metric	Meaning
TTFT	Time to first token. Strongly affected by queueing and prefill.
TPOT / ITL	Time per output token. Indicates decode steady-state behavior.
Throughput	Tokens/sec or requests/sec. Useful but can hide tail latency.
Goodput	Useful throughput under a latency target. Often better for online services.
P95 / P99	Tail latency. Important for multi-tenant and bursty workloads.

Metric set

A serving result is clearest when throughput, latency, tail behavior, and configuration are reported together.

Where latency is spent

Before the first token

- Queueing and admission.
- Tokenization and request validation.
- Prefill over the input context.
- KV allocation and graph path selection.

After the first token

- Decode iterations.
- Batch reshaping.
- KV page lookup and residency.
- Stream backpressure and tail behavior.

The same request contributes to different bottlenecks at different times.

Part II. Scheduling and State Management

Continuous batching, KV cache, and stateful execution

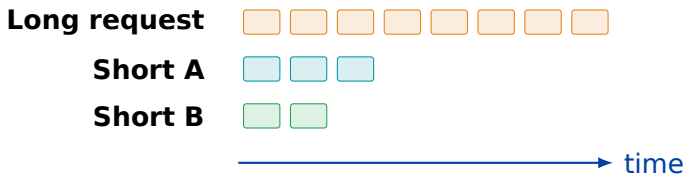
Why scheduling comes first

Scheduling tension

Online serving must continuously organize requests with very different lengths into efficient batches.

- Bigger batches improve device utilization.
- Bigger batches may also make new users wait longer.
- Long requests keep KV state alive for many iterations.
- Short requests need low waiting time even when long requests are active.

Head-of-line blocking in mixed workloads



- If the system waits for the whole request to finish, short requests may wait unnecessarily.
- Average throughput may look acceptable while TTFT and P99 degrade.

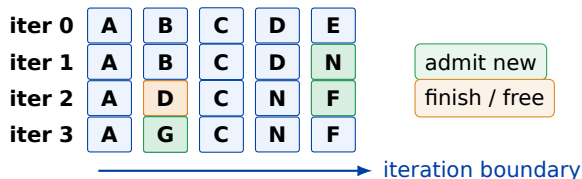
Request-level vs. iteration-level scheduling

Granularity	Benefit	Risk
Request	Simple implementation	Poor ability to insert new requests during generation
Iteration	Rebuild batch after each decode round	Needs careful state tracking and KV management
Token	Maximum flexibility	High metadata and synchronization overhead

Orca-style intuition

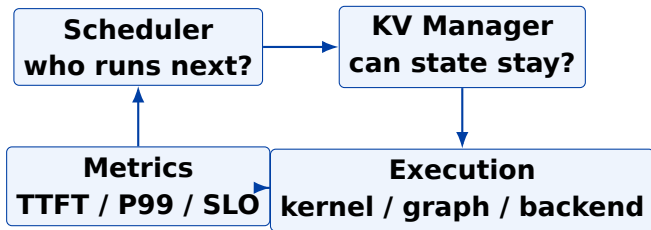
Move the scheduling decision from the request boundary to the generation-iteration boundary.

Continuous batching



- New requests can enter when old requests finish.
- The price is that every active request needs its own state tracked across iterations.

Scheduling is not isolated



Coupling point

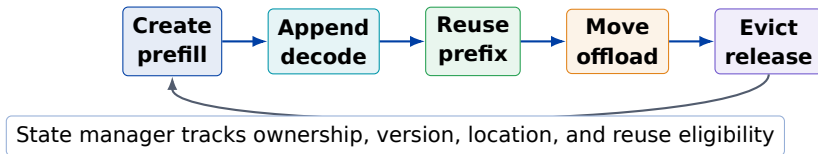
Scheduling decisions, KV residency, execution path, and service metrics form one feedback loop.

Why KV cache is central

- KV grows with context length and concurrency.
- It must stay resident across many decode iterations.
- It limits effective batch size.
- It turns every active request into a stateful object.
- Once prefix reuse is enabled, KV becomes a shared resource.

KV cache is not an implementation detail. It is a first-class system resource.

KV cache lifecycle



Design point

Each optimization changes one lifecycle step and may shift cost elsewhere.

Why contiguous allocation breaks down

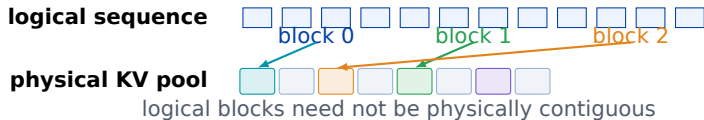
If KV must be contiguous

- Dynamic lengths create fragmentation.
- Reserving too much wastes memory.
- Reserving too little causes failures.

Paged idea

- Split KV into blocks.
- Map logical sequence to physical pages.
- Let the scheduler manage more dynamic requests.

Paged KV: a visual analogy

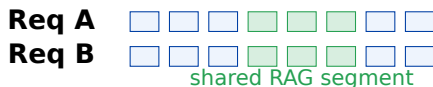


System effect

PagedAttention-style management reduces memory waste, supports dynamic batching, and creates a handle for reuse, migration, and multi-level KV storage.

Sharing KV: a small RAG example

- User A asks about a course policy.
- User B asks a different question about the same policy document.
- Both prompts include the same retrieved paragraph.



Opportunity: reuse the expensive state for the shared segment.

But sharing has a contract

Version, parameters, permissions, and semantic boundaries must match.

State decisions made by the runtime

Decision	Local benefit	System cost
Keep	Avoid recomputation	Occupies memory and limits new batches
Reuse	Save prefill work	Requires matching, validation, and ownership checks
Move	Release device memory	Adds transfer latency and coordination
Evict	Admit urgent work	May trigger recomputation later

Scheduling view

The scheduler is no longer choosing only which request runs next; it also

10-Minute Break

Session 2 resumes with modern workloads, accelerators, and evaluation.

Before the second session

Keep three objects in mind: requests, intermediate state, and evidence.

Part III. Modern Workloads and Heterogeneous Execution

Long context, RAG, MoE, agents, and accelerator dynamics

KV pressure changes scheduling

Scenario

The GPU/NPU memory is almost full. A new high-priority request arrives.

Option 1

Keep all KV, reject or delay the new request.

Option 2

Preempt one request and keep its KV elsewhere.

Option 3

Drop KV and recompute later.

Scheduling now includes a state-utility decision.

New forms of intermediate state

State form	Where it comes from	System challenge
KV blocks	Long context and continuous batching	Memory pressure, fragmentation, eviction
Prefix / segment	System prompts, RAG, templates	Matching, consistency, reuse cost
Draft state	Speculative decoding	Branching, rollback, commit
Agent state	Tool calls and multi-turn memory	Cross-step propagation, isolation

Research trend

Intermediate state is becoming a managed resource, not a temporary tensor.

Long context and RAG

- Longer prompts amplify prefill cost.
- RAG makes input length and overlap more variable.
- Similar documents create reuse opportunities.
- The benefit depends on matching cost, memory pressure, and latency targets.

Example

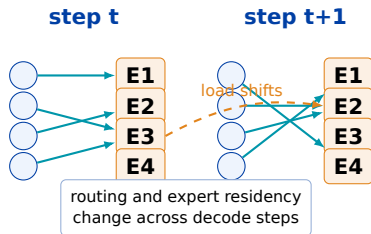
A university assistant may answer 100 different questions using the same handbook. Blindly recomputing the handbook prefix wastes prefill work.

MoE: routing is dynamic

What changes with MoE?

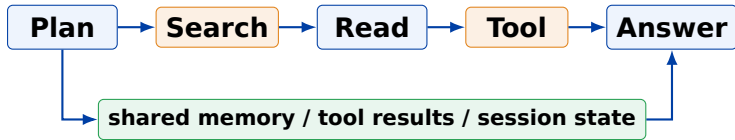
Each token may activate different experts, and the expert load can change from token to token.

- Expert load imbalance.
- Expert parallel communication.
- Expert offloading and residency.
- Dynamic expert-to-device mapping.



Challenge: dynamic expert mapping conflicts with stable graph replay requirements.

Agents: one request becomes a workflow



- Agent requests create intermediate state across multiple steps.
- Scheduling, caching, and isolation now span a workflow, not a single sequence.

Many reuse mechanisms, one selection problem

Selection space

APC, prompt cache, CacheBlend, LMCache, and other mechanisms may all help, but each has different cost and applicability.

- Timing of reuse attempts.
- Request and segment selection.
- Mechanism selection.
- Detection cost versus recomputation cost.

APC

Prompt Cache

CacheBlend

LMCache

Cost-based selector

Modern workloads change the control surface

Workload	New pressure	Control point
Long context	Heavy prefill and large KV	Prefix reuse, paging, admission
RAG	Shared but imperfect context overlap	Segment matching and validation
MoE	Dynamic expert load and residency	Routing, expert placement, graph path
Agents	Multi-step state and tool outputs	Workflow scheduling and isolation

Teaching point

Modern inference systems optimize a moving service workflow, not a single static model invocation.

Platform differences are system differences

- Operator libraries differ.
- Graph execution and compilation behavior differ.
- Interconnect and host-device paths differ.
- Profiling and debugging tools differ.
- Default precision and fallback behavior differ.

Takeaway

A GPU optimization is a hypothesis, not a guaranteed solution, on NPU/DCU/XPU platforms.

Three levels of adaptation

1. **Can run**: model loading, basic inference, correct output.
2. **Can optimize**: platform-aware kernels, graph paths, memory and communication.
3. **Can verify**: fair baseline, controlled workload, reproducible metadata.

Research usually starts at level 2

Running is necessary, but optimization and verification are where system research begins.

Graph replay wants stability; serving wants dynamics

High-performance execution wants

- Stable shapes
- Stable addresses
- Stable memory layout
- Predictable kernels

Online serving gives

- Variable prompt lengths
- Dynamic batching
- Changing KV pages
- MoE routing changes

This conflict is especially visible on graph-oriented accelerator stacks.

Measurement checklist

- End-to-end: TTFT, TPOT, P99, goodput.
- Runtime probes: graph hit rate, fallback count, memory allocation failures.
- Hardware probes: device utilization, HBM usage, communication time.
- Stability probes: variance across runs, tail behavior under bursty load.

Cause separation

Is the regression caused by scheduling, memory pressure, graph fallback, communication, or benchmark mismatch?

Part IV. Evidence and Open Engineering Practice

Benchmarking, reproducibility, and vLLM-HUST workflow

Why benchmark evidence is fragile

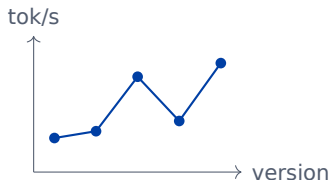
- Same workload name, different token distribution.
- Same model name, different precision or parallelism.
- Same code path, different backend defaults.
- Same throughput, very different tail latency.
- Same benchmark, missing metadata.

A benchmark is an evidence chain: workload, setup, baseline, and metrics.

Minimum metadata for a credible result

Category	Must record
Code	Repository, commit, PR, dependency versions
Model	Model ID, dtype, quantization, parallel strategy
Hardware	Device type, number of cards, driver, runtime stack
Service config	Max length, graph/eager mode, cache/offload settings
Workload	Input/output lengths, concurrency, arrival process, data source
Metrics	TTFT, TPOT, P99, throughput, goodput, memory usage

Example: a misleading performance trend

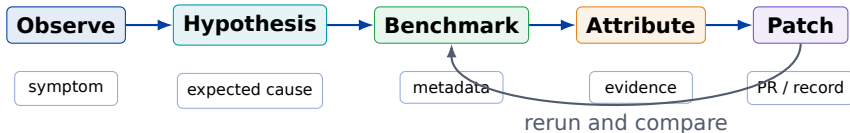


Consistency checks

- Same model?
- Same input/output length?
- Same dtype?
- Same graph mode?
- Same hardware and driver?

A large jump needs attribution: optimization effect or experiment change.

From observation to evidence



Evidence loop

A performance claim becomes useful only after the baseline, workload, configuration, and code path are recorded and rerun.

vLLM-HUST: putting system work into an open workflow

Practice target

vLLM-HUST, vLLM-Ascend-HUST, benchmark, and dev-hub repositories organize real inference-system development tasks.

- Start from an issue, CI failure, benchmark gap, or documentation problem.
- Record environment, code path, commands, and failure symptoms.
- Submit a small patch or standardized benchmark record.
- Use CI, review, and reruns to create maintainable evidence.

Hands-on practice path

- Define the system path being changed.
- Build a minimal reproduction.
- Compare against a clear baseline.
- Record metadata and known limitations.
- Produce a patch, PR, benchmark record, or documentation artifact.

Practice goal

Turn a system observation into a reproducible engineering claim.

Closing

Concepts, open questions, and further reading

Five concepts to remember

1. **Request lifecycle**: API, queue, prefill, decode, stream, release.
2. **Service metrics**: TTFT, TPOT, P99, throughput, goodput.
3. **Continuous batching**: dynamically reshape active batches.
4. **KV cache**: the central intermediate state and memory constraint.
5. **Heterogeneous execution**: hardware differences reshape bottlenecks.

Open research questions

- Global selection among KV reuse mechanisms.
- Boundaries for saving, sharing, or isolating agent intermediate state.
- How can MoE routing coexist with high-performance graph replay?
- Which GPU serving lessons transfer to domestic NPU platforms?
- What benchmark suite best represents bursty, long-tail, mixed workloads?

Recommended progression

1. Draw the lifecycle of one request.
2. Separate prefill from decode.
3. Study continuous batching and KV cache.
4. Read one real runtime: vLLM, SGLang, or TensorRT-LLM.
5. Reproduce one small benchmark with full metadata.

Starting point

A minimal reproducible experiment exposes the real system boundary and prevents accidental claims.

Recommended reading and systems

- Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI 2022.
- vLLM / PagedAttention: Efficient Memory Management for LLM Serving. SOSP 2023.
- SGLang: Efficient Execution of Structured Language Model Programs.
- LMCache, CacheBlend, Hydragen, and related KV reuse/sharing work.
- vLLM-HUST and vLLM-Ascend-HUST for domestic heterogeneous serving practice.

A useful reading lens: changed control point, introduced state, and demonstrated end-to-end value.

Summary

LLM inference systems build a verifiable service loop across dynamic requests, complex intermediate state, and heterogeneous execution paths.

- Scheduling absorbs dynamic load.
- KV cache determines memory pressure and reuse opportunities.
- Heterogeneous hardware reshapes the execution path.
- Benchmarking decides whether the story is trustworthy.

Thank you!

Questions and Discussion

Shuhao Zhang Huazhong University of Science and Technology